

HelixSpecifier

HelixSpecifier

Desarrollo basado en especificaciones que ajusta su propia ceremonia al trabajo.

Resumen

HelixSpecifier es un motor Go que fusiona tres metodologías de desarrollo —el flujo basado en especificaciones de SpecKit, la disciplina TDD de Superpowers y el ciclo de vida por hitos de GSD— en un único flujo adaptativo. Clasifica cada tarea según el esfuerzo requerido y ajusta la cantidad de proceso en consecuencia.

Descripción breve

HelixSpecifier es un motor de fusión para desarrollo basado en especificaciones, diseñado para agentes AI. Combina SpecKit, Superpowers y GSD, clasifica el trabajo por nivel de esfuerzo, ejecuta fases de especificación respaldadas por debates, impone una proporción mínima de pruebas por código y aprende de cada flujo completado.

Descripción detallada

HelixSpecifier es un motor de fusión para desarrollo basado en especificaciones (SDD, por sus siglas en inglés) escrito en Go (módulo `digital.vasic.helixspecifier`), desarrollado como componente del conjunto HelixAgent AI. Toma tres prácticas de desarrollo que normalmente existen en herramientas separadas — y en mentalidades distintas— y las fusiona en un único flujo de trabajo adaptativo: el proceso SDD de siete fases de SpecKit (Constitution, Especificar, Aclarar, Planificar, Tareas, Analizar, Implementar), la disciplina de desarrollo guiado por pruebas de Superpowers con ejecución paralela de subagentes, y la gestión

del ciclo de vida por hitos de GSD. Cada pilar sigue desempeñando su función; el motor es lo que los integra en un flujo coherente en lugar de una cadena ensamblada manualmente.

Su idea central es la *ceremonia adaptativa*: el motor clasifica el trabajo entrante según su nivel de esfuerzo y ajusta la cantidad de proceso para que una corrección de una línea no pase por el mismo ritual exhaustivo que una funcionalidad importante —ni una funcionalidad importante se apruebe con el rigor de un error tipográfico—. Diez características avanzadas se construyen sobre esta base: ejecución paralela de tareas con concurrencia limitada, un “Constitution como Código” legible por máquina que aplica reglas obligatorias de forma automática, “TDD Nyquist” que supervisa e impone una proporción mínima entre pruebas e implementación, debates multironda y multiagente para refinar especificaciones, aprendizaje adaptativo de habilidades y competencias, análisis de código heredado, generación predictiva de especificaciones a partir de patrones históricos, transferencia de conocimiento entre proyectos, ajuste de ceremonia en tiempo de ejecución que se reconfigura sobre la marcha, y una memoria persistente de especificaciones con búsqueda semántica.

Se consume como un módulo Go —mediante `go get` o una directiva de reemplazo local— detrás de un motor API deliberadamente minimalista: se registran los tres pilares junto con un escalador de ceremonia y una memoria de especificaciones, se clasifica el esfuerzo del trabajo y, a continuación, se ejecuta el flujo completo para obtener un resultado con puntuación de calidad. La interfaz es sencilla; la orquestación que hay detrás, no. Al igual que el resto de la familia Helix, se desarrolla bajo un régimen de verificación anti-engaño, con un ejecutor de desafíos en proceso que prueba código real en lugar de simulaciones.

Por qué lo creamos

El desarrollo basado en especificaciones, el TDD riguroso y la gestión por hitos suelen ser tres prácticas independientes con herramientas distintas. HelixSpecifier se creó para que un agente AI (HelixAgent) pueda ejecutar las tres como un flujo coherente y autoescalable, en lugar de tener que unirlos manualmente.

Por qué es un cambio de juego

Hace que el proceso sea proporcional al trabajo —de forma automática—. Los equipos suelen atascarse en uno de dos extremos negativos: ceremonias excesivas en todo (seguro pero lento, y con resentimiento silencioso) o ausencia de ceremonias (rápido hasta que deja de serlo). HelixSpecifier elimina esa disyuntiva ajustando la ceremonia al esfuerzo clasificado de cada

tarea y reajustándola en tiempo de ejecución a medida que el trabajo se revela. La capacidad que antes no era práctica es un proceso que se dimensiona por tarea —y, además, decisiones de especificación respaldadas por un debate multironda y multiagente con puntuación de posturas, en lugar de la primera suposición de un solo agente—.

Qué es innovador

- **Ceremonia adaptativa:** nivel de proceso impulsado por métricas de calidad en tiempo real y ajustado durante la ejecución, no fijado de antemano.
- **Nyquist TDD:** un umbral de proporción entre pruebas e implementación (mínimo 2x), inspirado en el teorema de muestreo de Nyquist: para capturar fielmente un comportamiento, hay que muestrearlo muy por encima de su frecuencia, por lo que las pruebas deben superar en cantidad al código que cubren.
- **Arquitectura de debate:** refinamiento de especificaciones en múltiples rondas y con múltiples agentes, donde se proponen, puntúan y convergen posturas, reemplazando una sola opinión por un enfoque adversarial.
- **Especificación predictiva y transferencia entre proyectos:** el motor extrae patrones de flujos acumulados para anticipar especificaciones y trasladar conocimientos valiosos de un proyecto a otro.
- **Constitution como Código:** reglas de proyecto obligatorias convertidas en legibles por máquina y aplicadas por el motor, no dejadas al criterio de los revisores.

Principales desafíos técnicos y cómo los resolvimos

- **Fusionar tres metodologías sin que compitan entre sí:** SpecKit, Superpowers y GSD asumen que dominan el flujo de trabajo. Se resolvió con un motor de fusión que registra cada pilar tras una interfaz común y los impulsa a través de un ciclo de vida compartido, de modo que se integran en un solo proceso en lugar de tres que chocan.
- **Determinar cuánto proceso necesita realmente una tarea:** si se sobrestima, todo se ralentiza; si se subestima, el trabajo arriesgado se entrega sin revisión. Se resolvió con un clasificador de esfuerzo que dimensiona el trabajo y alimenta un escalador de ceremonias que ajusta el nivel de proceso dinámicamente a medida que avanza la ejecución.
- **Mantener alta la calidad de las especificaciones sin un guardián humano en cada decisión:** se resolvió reemplazando las especificaciones de un solo intento por un

refinamiento respaldado por debates, donde los agentes puntúan posturas en competencia a lo largo de varias rondas, y aplicando las proporciones de Nyquist TDD para que la implementación no supere a sus pruebas.

Pila tecnológica

- **Go:** elegido para que el motor se distribuya como un binario importable sin dependencias en tiempo de ejecución; su modelo de concurrencia es lo que hace viable el despacho de tareas en paralelo con límites y las rondas de debate multiagente, en lugar de convertirse en un dolor de cabeza de hilos.
- **logrus:** registro estructurado integrado en el motor y los tres pilares, de modo que las decisiones de un flujo (clasificación, cambios de ceremonia, resultados de debates) sean legibles a posteriori.
- **Pilar SpecKit:** el proceso de desarrollo guiado por especificaciones en siete fases (Constitution → Especificar → Aclarar → Planificar → Tareas → Analizar → Implementar), que proporciona la columna vertebral disciplinada de cómo una especificación se convierte en código.
- **Pilar Superpowers:** disciplina TDD con ejecución paralela de subagentes, que aporta el rigor de pruebas primero y la distribución que mantiene honesta y ágil la implementación.
- **Pilar GSD:** gestión de hitos y ciclos de vida, que da al flujo su noción de “hecho” y su progresión a través de etapas.
- **Almacén de memoria de especificaciones:** un índice persistente y semánticamente buscable de especificaciones pasadas, el sustrato que hace posible la especificación predictiva y la transferencia entre proyectos, en lugar de empezar desde cero cada vez.

Notas sobre estado y honestidad

- **Estado: beta.** Consumido como componente del módulo Go de HelixAgent.
- **Licencia: por determinar.** No se detectó ninguna LICENCIA mediante GitHub API — SIN VERIFICAR / no declarada.
- El nombre para visualización “HelixSpecifier” se asigna al repositorio specifier.

Nivel de prioridad: Helix-principal.